CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Learnings and Reflection Report

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Team 40 — Sajjad Nakhwa (23B0702) · Dependar Arya (23B0331) · Neel Shah (23B4244) · Akshat Karkar (23B4229) · Pearsh Gumasta (23B4232)

Contents

1. Subject learnings
2. AI capability assessment
3. Validation insights
4. Practical relevance
5. Individual reflections
6. Ethics and limitations

1. Subject learnings

1.1 The third number

Before this project our mental model of project cost control was the one most people carry: compare what you have spent against what you planned to spend, and the difference is the problem.

The case study broke that model in a way a lecture had not. At the data date the project had spent ₹3.98 crore against a planned ₹4.02 crore. By the conventional reading it is marginally **under** budget and the report writes itself. It is in fact both significantly overspending and significantly behind programme, and the apparent health is a coincidence of two errors pointing in opposite directions — less work done than planned, each unit of it costing more than planned.

The insight is small to state and large in consequence. **Two numbers can only produce one comparison.** Budget versus spend conflates "am I doing the work" with "is the work costing what it should", and there is no way to separate them without a third measurement. Earned value — the budgeted cost of the work actually performed — is that third number, and everything else in EVM is bookkeeping on top of it.

What surprised us is how cheap the third number is. It requires no new cost system. It requires someone to state, per work package, what fraction is complete. The barrier to EVM in Indian construction practice is not computational; it is that nobody is made accountable for that fraction.

1.2 Indices are not the point; the forecast is

We began thinking CPI and SPI were the output. They are not. They are intermediate quantities whose value is almost entirely that they feed a forecast.

A CPI of 0.872 means nothing to a client. "At current performance this project finishes at ₹16.06 crore against a budget of ₹14.00 crore" means a great deal, because it is a number someone must now find or write off. The index matters because it supports that sentence.

This changed the tool. An early version led with a grid of indices. The final version leads with a plain-language paragraph and puts the forecast in the third tile row, because that is the order in which the questions are actually asked.

1.3 The four EAC methods, and why there is no right one

We expected to learn "the" formula for estimate at completion. There isn't one, and understanding why was among the more valuable things in the project.

The four standard methods differ in exactly one assumption: what you believe about the cost performance of the work that remains.

Method	Belief	Case study result
EAC1 = BAC / CPI	Performance to date is typical and continues	₹16.06 Cr
EAC2 = AC + (BAC − EV)	The variance was a one-off; the rest runs at budget	₹14.51 Cr
EAC3 = AC + (BAC − EV)/(CPI × SPI)	Cost *and* schedule pressure continue	₹17.98 Cr
EAC4 = weighted	Cost dominates; schedule partly weighted	₹16.09 Cr

The spread is **₹3.47 crore on a ₹14 crore project** — a quarter of the contract value. That spread is not a defect in the method; it is the method being honest. Narrowing it requires a judgement nobody can make arithmetically: was the rock excavation a closed event, or a symptom of a site that was never properly investigated?

Two mathematical observations clarified the family for us. First, **TCPI against EAC1 equals CPI exactly** — a forecast built on "performance continues" demands, by construction, no change in performance. It is a tautology, and recognising it is what shows EAC1 is a *projection*, not a *plan*. Second, **VAC on EAC2 equals the cost variance already incurred** — if the remainder runs at budget, the final overrun is precisely the overrun to date. Both identities now sit in the test suite, where they also serve as a check that the whole chain is intact.

Our conclusion, which we would defend in practice: **report the range.** A single EAC implies a precision the method does not possess.

1.4 TCPI is the sharpest instrument in the set

The most useful single number we encountered was the one we knew least about beforehand.

TCPI = (BAC − EV) / (BAC − AC) is the cost efficiency the remaining work must achieve to land on budget. On the case study it is **1.051**, against a CPI the project has sustained at **0.872** for six consecutive periods. Finishing on budget requires a **20.6% improvement** on performance never once achieved.

What makes it powerful is that it converts an argument into an arithmetic statement. A recovery plan claiming the budget can still be met is now claiming something specific and falsifiable. In a room where a contractor is presenting a recovery plan and a client is deciding whether to believe it, TCPI is the question that ends the discussion.

We also learned when it stops meaning anything: once AC reaches BAC the denominator is zero or negative, and there is no efficiency at which the remaining work can be done within a budget that is already gone. The engine returns NaN rather than a number, which we think is the honest behaviour.

1.5 Earned Schedule, and a defect we would not have noticed

The most substantial piece of subject learning came from a question we asked almost in passing: why does SPI behave oddly near the end of a project?

The answer is structural. SPI = EV / PV, and at completion EV must equal BAC and PV must equal BAC. **Therefore SPI returns to 1.00 on every project ever completed, however late.** Schedule variance returns to zero. The indices improve through the final third and the improvement is an artefact of the arithmetic. A planner watching SPI receives their most reassuring reading at the moment the project is latest.

This is not a subtlety; it is a hole in a measure that appears in every textbook and most contracts. Earned Schedule fixes it by asking when on the baseline the current earned value was scheduled to have been earned, which yields a **time**:

```
ES     = 5.357 months of baseline value earned
AT     = 6 months elapsed
SV(t)  = −0.64 months  ≈ 19 days behind
SPI(t) = 0.893
```

"About nineteen days behind programme" is actionable. "SPI is 0.863" is not. We implemented both and display both, with the gap explained, rather than quietly preferring one — partly because practitioners know SPI and would distrust a tool that omitted it, and partly because the disagreement between them is itself informative.

1.6 The empirical regularity that changes how you read a trend

The literature reports that CPI tends to stabilise early in a project — often by the 20% complete point — and rarely recovers thereafter.

We built the case study to exhibit this, and seeing it plotted taught us more than reading it. Across six periods CPI sits between 0.86 and 0.88 while SPI swings from 0.88 down to 0.74 and back to 0.86. Schedule performance is volatile; cost performance is not.

If that regularity holds, a flat below-1.00 CPI is strong evidence against any recovery plan that assumes cost performance will improve. Combined with TCPI, it gives a reviewer a defensible basis for scepticism rather than a hunch.

We are careful to report this as a **finding from the literature that our synthetic case was built to illustrate**, not as something this project has independently verified. One constructed dataset demonstrates nothing empirically. That distinction is exactly the kind we were least careful about before this project.

1.7 Aggregation destroys the diagnosis

The project's main functional decision was to carry the work breakdown structure through every calculation rather than aggregating to a single project curve.

A project CPI of 0.872 says something is wrong. The breakdown says: **₹38 lakh of the ₹51.20 lakh adverse variance sits in three substructure packages, all complete, so that portion is final and no recovery plan touches it**; only ₹7.7 lakh, in the frame packages still running, is live.

Those are different conversations. The first invites a generic cost-control exercise. The second says: stop trying to recover the substructure, protect the frame, and tell the client ₹38 lakh is gone.

The prior-art tool we studied rolls each project to a single percentage curve and lists the absence of a WBS breakdown as a known limitation. We now understand why that limitation is the significant one.

---

2. AI capability assessment

The project was built with Claude (Anthropic) throughout. Full log in `docs/05-ai-usage-log.md`. This section assesses capability honestly, with specific examples in both directions.

2.1 Where AI was genuinely strong

Operational knowledge that would have cost days. Three examples. That openpyxl writes formulas but no cached results, so a workbook read back appears empty unless `fullCalcOnLoad` is set. That LibreOffice can be driven headlessly to evaluate a workbook, and needs a dedicated user profile or it exits silently when another instance is open. That Excel treats an absent `invertIfNegative` as *true* and fills negative bars white, so an all-negative cost variance chart renders as hollow outlines.

Each of these is the kind of thing that costs an afternoon of confused debugging and appears in no textbook.

Formula recall, accurate. Every EVM formula it produced was correct when checked against the standard definitions, including the less-used ones — TCPI against EAC rather than BAC, the weighted EAC4, the Earned Schedule interpolation. We checked all of them. None was wrong.

Arithmetic verification beyond our patience. When our hand calculation of EAC3 disagreed with the engine by ₹156, it recomputed the quantity in 40-digit decimal arithmetic and demonstrated that the engine matched full precision and our hand figure had rounded mid-chain. We would have accepted the ₹156 as rounding noise and moved on, and would have missed a rule worth adopting.

Adversarial review of its own output. Asked to review the progress carry-forward routine it had written an hour earlier, it identified a nested loop indexing a pivot table by a column that might not exist. The defect had not yet produced a wrong answer, because the case study has no gaps in reporting — but real project data does.

Arguing against its own use. Asked whether the monthly commentary should call a language model, it argued against, on grounds we adopted (§2.4 below). We note this is also the kind of answer a model might give because it reads as appropriately cautious, which is why we evaluated the argument on its merits rather than on its source.

2.2 Where AI was wrong or needed correcting

It does not know what matters. Its first draft of the validation report opened with "50 of 50 comparisons agree" and relegated the limitations to an appendix. But the more important finding is that both implementations read the same inputs, so **neither can detect a wrong percent-complete figure**. We restructured the report to lead with the limitation. It did not propose this, and would not have.

This is the central limitation. It optimises for the question asked. It does not ask whether the question is the right one.

Domain idiom. The commentary described a cost index as "behind plan". A cost index is never behind; schedule is behind, cost is above or below. Small, and exactly the kind of thing that signals to a practising engineer that the author has not worked on a site.

Case study realism. Its first draft spread cost variance evenly across all work packages. Arithmetically fine, and it would have made the project's central feature — locating variance — pointless, because everything would have been equally to blame. It optimised for "produce a case study with a variance" rather than "produce a case study that demonstrates why this tool exists".

Anything visual. The dashboard's executive summary rendered white-on-white under a dark browser theme. Every number correct, all 128 tests passing, tool unusable. Found by opening it and looking.

It generated the mis-constructed tests too. Two validation tests were written in a way that triggered an earlier rule than the one under test. It also diagnosed them correctly when asked. The pattern is consistent: good at producing plausible work, good at critiquing work when directed to, not reliable at noticing unprompted that its own output is wrong.

2.3 A specific instance of wrong-in-a-plausible-way

The most instructive error was ours, caught by it — but it illustrates the risk precisely.

Had the EAC3 discrepancy been ₹156 in the *other* direction — had the engine been wrong and our hand calculation right — would we have caught it? Only because we had a hand calculation at all. A team that had accepted the engine's figure, formatted it to two decimal places and put it in a report would have had a wrong forecast presented with six-figure precision and complete confidence.

That is the characteristic risk of AI-generated analysis: not that it is obviously wrong, but that it is wrong in a way that looks exactly like being right. EVM output is particularly exposed — three ratios near 1.00, no intuition a reader can apply. A CPI of 0.94 from corrupt data looks exactly like a CPI of 0.94 from good data, and it will be taken to a client meeting.

2.4 The decision not to use AI in the product

The brief asks for "automatic plain-language monthly progress commentary". The obvious implementation passes the indices to a language model. We decided against it, and the decision is the project's main argument about AI in professional practice.

Three reasons:

1. **Reproducibility.** The same inputs must produce the same sentence every month. A cost report is defended line by line, and "the model phrased it differently this time" is not a defence.
2. **It cannot invent.** A language model handed a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In `evm/commentary.py` no figure can reach the text without having been computed, and a test asserts it.
3. **Auditability.** Every sentence traces to a threshold written down and justified in one file. A reader who disagrees with "within 5% is noise" can find it and argue.

So the tool is *less* AI-powered than it could be, and that is the point. **The right division is that AI belongs upstream of the number, not between the number and the reader.** It drafted the templates, checked the formulas, wrote the code and caught our arithmetic slip. It does not generate the sentence a client acts on.

2.5 What it changed about how we work

The most durable change is a habit: **ask the question that could prove it wrong, and ask it in a different system.**

The validation strategy came from a question of this kind — "how would we know if we had misunderstood the formula?" — and the answer, that a Python re-implementation would embed the same misunderstanding, is why the benchmark is executed in a spreadsheet engine instead. That habit is not about AI at all. It is ordinary engineering discipline, which AI makes more necessary because it makes plausible output so much cheaper to produce.

---

3. Validation insights

3.1 What the comparison showed

Fifty comparisons across two reporting periods, agreeing to ₹0.01 on currency and 1×10^{-9} on dimensionless indices. Two independently written implementations — Python and typed Excel formulas — of the same definitions.

3.2 The insight that reshaped the strategy

Our first plan was a second Python implementation to compare against.

The argument against it, from the prior-art repository's documentation, is that *a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.* If we had misread a formula, the re-implementation would embed the same misreading, agree perfectly, and the validation would pass while both were wrong. The test would measure our consistency, not our correctness.

So the benchmark lives in a different execution environment with different semantics, and it is **executed, not re-implemented**. LibreOffice loads the workbook, recalculates every cell, and we read what a real spreadsheet engine computed.

The general principle, which we expect to carry into professional work: **a check is only worth what is independent about it.** Two implementations by the same person, in the same language, from the same understanding, are one implementation run twice.

3.3 What the discrepancies taught us

Four surfaced. Details in `docs/04-validation-report.md` §5.

The ₹156 EAC3 difference taught us that rounding is not cosmetic. Rounding CPI × SPI to six decimal places mid-chain moved an ₹18 crore forecast by tens of rupees. Immaterial here; the same habit applied to CPI early in a project moves it by lakhs. Rounding is for presentation only.

Two mis-constructed tests taught us the discipline of asking *why* a test fails before changing anything. Both failures were in the tests. The quick fix would have deleted a correct validation rule — the one rejecting progress booked against a summary WBS branch, which is a real way for earned value to be double-counted.

The dark-theme bug taught us that "validated" and "works" are different claims. Every number correct, every test passing, screen unreadable. Only one of those claims can be automated.

The carry-forward routine, caught in review rather than by a test, taught us that a defect invisible on clean data is exactly the defect that surfaces on real data. The case study has no gaps in reporting. Real projects do.

3.4 The most important thing validation did not do

This is the finding we would lead with if we could only report one.

The benchmark validates the calculation. It cannot validate the input.

Both implementations read the same budgets, the same percent-complete figures, the same costs. That is deliberate — a benchmark against different data would compare nothing. But it means that if a work package is reported at 60% complete when it is really 40%:

- the Python engine computes a wrong earned value,
- the Excel benchmark computes **the same** wrong earned value,
- all 50 comparisons agree perfectly,
- every index on the dashboard is confidently, precisely wrong,
- and nothing in the software can detect it.

Earned value is percent complete multiplied by budget. Percent complete is a judgement made at site, often by a person with an interest in the answer. A contractor's engineer assessing their own progress ahead of an interim payment application is not a neutral observer.

The practical consequence is that the governance around how percent complete is assessed, by whom, and with what independent check, matters more than the software. It is also the part most likely to be neglected, because it is the part that cannot be bought, installed or demonstrated in a fifteen-minute presentation.

We find this genuinely uncomfortable, because we have spent the project building the part that does not matter most.

---

4. Practical relevance

4.1 Would anyone use it?

A contractor's planning engineer: probably yes, with changes. The work-package breakdown and the exported live-formula workbook are directly useful. The blockers are integration — the tool wants three clean CSVs, and a real planning engineer has a Primavera P6 export and a Tally cost ledger that do not share a coding structure. Reconciling those is most of the real work and the tool does none of it.

A client's project management consultant: yes, with the least change. This is the best fit. A PMC reviewing a contractor's monthly report wants exactly what the tool provides: an independent recomputation of the indices, a view of where variance sits, and TCPI as a test of whether the recovery plan is arithmetically possible. A PMC already receives progress data in summary form, so the integration problem is smaller.

A lender's independent engineer: yes, for the forecast range. The question is whether sanctioned debt will cover the project. The four-EAC spread and TCPI answer it directly, and the honest presentation of a range rather than a point estimate is closer to how a lender thinks.

A site engineer: no. Wrong altitude. They need to know which pour is late this week, not that CPI is 0.872.

4.2 What would have to change for real use

In rough order of how much each would block adoption:

1. **Schedule integration.** The tool reads a time-phased budget, not a network. It can say the project is 19 days behind in aggregate. It cannot say whether the slipped package is on the critical path, and therefore cannot say whether the completion date has moved. For a contractor arguing an extension of time this is the gap that matters most, and closing it means ingesting a P6 or MS Project file.
2. **Re-baselining.** One baseline per project. Real projects are re-baselined after major variations, and a tool that cannot represent that will be abandoned at the first one.
3. **Variations and claims.** Scope changes are assumed already incorporated. In practice a variation's cost is known long before its budget is formally approved, and projects spend months in a state where work is being done against a budget that does not yet include it.
4. **Cost ledger integration.** Manual CSV preparation each month will not survive contact with a busy site.
5. **Retention, advance and cash flow.** EVM measures cost and progress, not liquidity. A project can be green on every index here and still fail because it cannot pay subcontractors this month. In Indian construction this is the more common failure mode.
6. **Multi-project roll-up.** A PMC or a lender monitors a portfolio.
7. **Audit trail.** Who reported 22% complete on the columns, when, and on what basis. Given §3.4, this may matter more than anything else on this list.

4.3 The organisational obstacle

Everything above is solvable with engineering time. The real obstacle is not technical.

EVM requires someone to state, monthly and in writing, what fraction of each work package is complete — and then be held to it when the figure is compared against cost. That is a transfer of accountability, and it is resisted for exactly that reason. It is more comfortable for everyone if progress stays qualitative ("substructure is nearly done") because qualitative progress cannot be shown to have been wrong.

A tool makes the arithmetic free. It does not make the accountability free, and the accountability was always the expensive part. We think this explains why EVM adoption in Indian construction is lower than the obvious benefit would predict, and it is not something better software will fix.

4.4 Testing with a practitioner

The brief offers bonus credit for testing with a practising engineer. We have not been able to do this within the timeframe.

What we would ask, were the opportunity available, and we would structure it to seek disconfirmation rather than approval:

1. Show the dashboard without explanation and ask what they conclude. (Tests whether the plain-language summary works.)
2. Ask how they currently assess percent complete, and who checks it. (Tests §3.4 against practice.)
3. Show the TCPI of 1.051 against CPI of 0.872 and ask whether they would accept a recovery plan claiming the budget is still achievable.
4. Ask what in the monthly commentary they would delete before sending it to a client.
5. Ask what would stop them using this on Monday.

We expect question 2 to produce the most useful answer and question 5 the most deflating one.

---

5. Individual reflections

Each entry below is written by the member named, against a common set of four questions, and covers the work that member owned.

5.1 Sajjad Nakhwa (23B0702) — Project Lead and Finance Modeller

My role: I owned the EVM method itself — which measures the tool would report, which forecasting formulas it would use, and the conventions for signs, rounding and undefined values that every other part of the project then had to follow. I also set the sequence of work and arbitrated the design arguments, of which the commentary question was the largest.

What I did not understand before: that there is no single EAC formula. I came into this expecting to learn *the* way to forecast a project's final cost, and instead found four standard methods that differ in one assumption each — whether the variance to date will persist, whether it was one-off, whether schedule pressure will also cost money. On our case study they span ₹3.47 crore. I initially read that spread as the method failing. It is the method being honest: the spread *is* the uncertainty, and collapsing it to one number does not remove the uncertainty, only the reader's awareness of it.

What I got wrong: I specified early on that the tool should select the most appropriate EAC and display a single figure, with the others available on request. That would have been the most confidently misleading feature in the project — a number carrying an implied precision the underlying method cannot support. I was argued out of it, correctly, and the dashboard now presents the range with each method's assumption stated beside it.

The second thing I got wrong was leaving the arithmetic conventions unwritten. Rounding, the sign convention on variances, and what an index should return when its denominator is zero are boring decisions, so I deferred them. We then spent real time on a ₹156 disagreement between a hand calculation and the engine that existed only because nobody had said where rounding was permitted. The rule we ended on — rounding is presentation only, never intermediate — should have been written on day one.

What I would do differently: fix the conventions before any code, and write them in one place that the engine, the benchmark and the hand calculations all cite.

What I will carry into practice: TCPI read against CPI as the first question to ask of any recovery plan. Our project needs 1.051 efficiency on remaining work having achieved 0.872 so far. That comparison turns "we will recover in the second half" from an assertion into a claim with a number attached, and it takes ten seconds to make.

5.2 Dependar Arya (23B0331) — Data Lead

My role: I owned the case study — the work breakdown structure, the budget for each of the 19 packages, the baseline window each package sits in, the time-phasing within those windows, and the actual cost and percent-complete figures for the six reported periods. I also compiled the data source list and the statement of what our figures are and are not.

What I did not understand before: how much of EVM's credibility rests on the baseline being internally consistent, and how quietly it fails if it is not. The time-phased budget has to sum to each package's budget, and the package budgets have to sum to BAC. If they do not, PV is wrong by the error, every schedule index is silently scaled by it, and nothing visible breaks — the dashboard still draws a plausible S-curve and still reports an SPI to three decimals. I had assumed errors in a model would announce themselves. The reconciliation check that now refuses to load a project whose baseline does not tie out is there because of this.

What I got wrong: my first version of the case study spread the cost overrun evenly across all 19 packages. It gave a realistic-looking project CPI and I was pleased with it. It also made the tool's central feature pointless: if every package is equally to blame, locating the variance tells a manager nothing he did not already know from the project total. The case study had to be rebuilt so that the variance is *concentrated* — 74% of it sits in three substructure packages, and those three are already complete, which is a different management problem from the same rupees sitting in live work. The lesson is that a dataset built to demonstrate a tool has to be built around the decision the tool is supposed to support.

What I would do differently: write the project's narrative first, in words, and derive the numbers from it. I worked the other way round — numbers first, then a story that would fit them — and spent longer reverse-engineering a plausible cause for figures I had already chosen than it would have taken to start from "hard rock in the excavation".

What I will carry into practice: that a dataset constructed to illustrate something proves nothing empirically, and has to be labelled as illustrative every single time it is shown. Ours is synthetic, every figure in it is assumed, and we say so in the charter, the method note, the manual, the workbook and on the dashboard itself. A number on a screen acquires authority it has not earned unless something on the same screen is actively denying it.

5.3 Neel Shah (23B4244) — AI and Tool Developer

My role: I owned the implementation — the Python calculation engine, the Streamlit dashboard, the Plotly charts, the live-formula Excel export, and the repository itself.

What I did not understand before: how much professional practice lives in defaults that nobody documents. Two examples cost me real time. `openpyxl` writes formulas into a workbook but writes no cached results for them, so a reader that trusts the cache — which includes a great deal of software — sees a workbook full of empty cells; the fix is one flag, `fullCalcOnLoad`, and nothing warns you that you need it. Excel fills negative bars in a bar chart with white unless explicitly told otherwise, so a variance chart renders with the adverse packages invisible. Neither is a bug in my code. Both produce a file that is

correct and unusable. Writing code that produces a correct file turns out to be a different problem from writing code that produces a file which *opens* correctly, and only the second one matters to the person receiving it.

What I got wrong: the dark-theme CSS bug, which is the failure I think about most. I styled the executive summary panel with a light background and did not set a text colour. Under a browser set to dark mode Streamlit supplied white text, and the single most important paragraph in the tool — the plain-language verdict a non-specialist reads first — rendered white on white. Every one of our tests passed. Every number was correct. The tool was unusable, and I found it only because I opened the running app to take screenshots for the manual. A test suite is evidence about arithmetic; it says nothing whatever about whether a human being can read the output.

What I would do differently: look at the running thing earlier and far more often. For most of the build my only feedback was pytest output, which is fast, precise and blind to everything I most needed to know.

What I will carry into practice: when generated code uses a library's defaults, check what those defaults *are* rather than assuming silence means sensible. The three chart overrides we adopted from the prior-art repository would each have taken me a day to find on my own, and I would have found them by someone telling me the output looked wrong.

5.4 Akshat Karkar (23B4229) — Validation Engineer

My role: I owned the check — the independent manual benchmark workbook, the hand calculations for Periods 5 and 6, the 150-test suite, and the discrepancy log.

What I did not understand before: that a check is worth only what is genuinely independent about it. My instinct, and my first plan, was to write a second implementation of the EVM formulas in Python and compare the two. That feels rigorous and is nearly worthless: both implementations would have been written by the same people from the same reading of the same formulas, so a formula I had misunderstood would have been misunderstood identically on both sides. The comparison passes, I report agreement, and both numbers are wrong. The benchmark therefore lives in typed Excel formulas acting on the raw inputs, and it is *executed* by LibreOffice rather than re-implemented — a different formula language, evaluated by a different engine. That one decision is the difference between a validation and a reassurance.

What I got wrong: my hand calculation of EAC3, which disagreed with the engine by ₹156. I assumed the engine was wrong, because the engine was the thing under test and I was the control. It was not wrong. I had rounded CPI × SPI to six decimal places partway through the chain, because that is what one does on paper, and the error propagated through the division. Recomputing at 40 significant digits showed the engine had been right throughout. Two things follow: precision rules have to be agreed before anybody calculates anything, and the control is not automatically correct merely because it is the control.

I also got two of my own tests wrong. Both asserted that a specific input-validation rule would reject a bad project, and both failed — because an *earlier* rule caught the same bad project first. The quick fix was to loosen the engine until the tests passed. That would have deleted a correct rule, one that catches earned value being double counted. I rewrote both tests to exercise each rule in isolation and changed no engine code.

What I would do differently: decide the precision and rounding rules before producing a single hand figure.

What I will carry into practice: ask *why* a test fails before changing anything at all. Two of our four discrepancies were defects in the check rather than in the thing being checked, and in both cases the fast fix would have made the project worse while making the output greener.

5.5 Pearsh Gumasta (23B4232) — Documentation and Reflection Lead

My role: I owned everything written — the user manual, the AI usage log, this report, and the presentation and viva preparation.

What I did not understand before: how much a document's structure is an argument, independently of its contents. The first draft of our validation report opened with "50 of 50 comparisons agree" and placed, in an appendix, the fact that neither implementation can detect a wrong percent-complete figure at source. Every sentence was true. The document said the tool was verified. Moving that limitation into a prominent numbered section, with nothing else changed, turns the same facts into a document that says the arithmetic is verified and the inputs are not. A reader who skims the first page and stops gets the opposite impression from the same evidence depending on an ordering decision I made without thinking about it.

What I got wrong: I nearly let the AI usage log become a list of 28 "Accept" entries. It would have been accurate, in that most AI output was in fact accepted, and it would have been evidence that nobody was checking anything. The entries actually worth reading are the four where something went wrong: where the AI was wrong, where it was right and we were wrong, and where we rejected output that would have worked perfectly well — the language-model commentary — on principle. I restructured the log to surface those four and said in the declaration why a uniformly positive log should not be believed.

What I would do differently: write the limitations section first rather than last. It is the section that gets written at the end, when everyone is tired and the tool is working and nobody wants to dwell on what it cannot do — and it is the first section a practising engineer turns to. Writing it first would also have improved the tool, because several limitations are things we could have addressed had we noticed them in week one instead of week four.

What I will carry into practice: repeat the most important caveat rather than stating it once. The percent-complete limitation appears in five places in our documentation, deliberately, and I expected that to feel repetitive. It does not, because a polished interface works continuously to make the reader forget it, and one mention in a method note is no match for a dashboard with a confident green verdict on it.

---

6. Ethics and limitations

6.1 Data reliability

Every figure in the case study is **assumed**. Budgets are order-of-magnitude plausible for an institutional building built to CPWD DSR-style item rates; they are not taken from any tender, estimate or invoice, and no claim is made that they are accurate unit rates.

This is stated in the charter, the method note, the data generator's docstring, the dashboard's data tab and the exported workbook. The reason for the repetition is that a polished dashboard confers unearned

authority. A number rendered in a large clean typeface next to a green tick reads as a fact, and ours are not facts.

The decision to use synthetic rather than real data was deliberate (method note §2.2) and it has a cost: we cannot claim the tool has been tested against the messiness of real data — missing periods, retrospective corrections, packages whose scope changed, costs booked to the wrong code. Stating what our validation does not cover is part of the validation.

6.2 Over-reliance on AI

The project was built with substantial AI assistance and we think the result is better for it. The risk is specific and worth naming precisely.

AI makes plausible output cheap. Writing a wrong EVM engine used to take as long as writing a right one; both now take minutes. The distinguishing effort has shifted almost entirely to verification, and verification is the part that is tedious, unglamorous and easy to skip when the output looks finished.

Our structural defence was to make verification independent: a manual benchmark in a different execution environment, hand-calculated expected values written before the engine ran, algebraic identities that hold only if the whole chain is intact. None of that is novel engineering. It is ordinary discipline, which AI makes *more* necessary rather than less.

The honest limit of our defence: it catches arithmetic and logic errors. It does not catch **errors of framing** — building the wrong thing correctly. Nothing in our test suite would have told us if EVM were the wrong method for this problem, and an AI optimising for the question asked will not raise its hand about the question.

6.3 Accountability for decisions made on AI-generated numbers

If a client cancels a contract on the strength of a forecast this tool produced, and the forecast was wrong, who is responsible?

Not the tool, and not the AI that helped write it. The engineer who signed the report. That is unchanged by the method of production, and it is worth stating because the fluency of AI-generated analysis invites a diffusion of responsibility that the law and professional practice do not recognise.

Three practical positions we adopted:

Traceability. Every number in every deliverable can be traced to an input and a formula. The benchmark workbook shows the arithmetic in visible cells. The commentary cannot contain a figure that was not computed. Someone who must defend the output can see how it was produced.

Refusing to compute rather than computing from bad data. The tool rejects inconsistent input rather than proceeding. This is deliberate and sometimes inconvenient. EVM output is three ratios near 1.00 with no sanity check a reader can apply; a CPI of 0.94 from corrupt data looks exactly like a CPI of 0.94 from good data. Refusing to produce a number is safer than producing one that cannot be challenged.

Disclosure. The AI usage log is a deliverable, not an appendix. It records where AI was wrong, where we were wrong, and where its output was rejected on principle.

6.4 The ethics of a convincing interface

The uncomfortable conclusion of this project is that we have spent most of our effort on the part that matters least.

The arithmetic is validated to nine decimal places. The arithmetic was never the risk. **The risk is the percent-complete figure typed into a spreadsheet by someone who will be paid more if it is higher**, and we have built an instrument that renders that figure with six-figure precision, a traffic-light verdict and a chart.

Every design choice that makes the dashboard more persuasive makes this worse. A tool that looked crude would be trusted in proportion to its reliability. Ours looks authoritative, and authority is exactly what it has not earned, because it cannot see its own inputs.

Our response was to put the limitation where it cannot be missed — on the dashboard, in the manual, in the charter, in the exported workbook, and leading the validation report's conclusions. That is a partial answer and we know it. A user in a hurry will read the number and not the caveat.

If there is one thing we would want a reader of this report to take away, it is that the quality of an EVM system is the quality of its percent-complete measurement, and everything else — including all of the work in this repository — is arithmetic on top of a judgement made by a person standing on a site.

6.5 Scope limitations, collected

Limitation	Consequence
Percent complete is an unverifiable site judgement	**All indices inherit its error. The governing limitation.**
No critical path or network	Cannot say whether a slip moves the completion date
No re-baselining	Unusable after a major variation
No retention, advance or cash flow	A project can be green here and still run out of money
Variations assumed already in the budget	A real gap between work done and budget approved
Equal-length periods assumed	Unequal periods distort the trend charts
Single project	No portfolio roll-up
Synthetic data only	Untested against real ledger messiness
Nominal rupees, one price base	No escalation
Chart legibility not machine-checkable	Needs a human to open the file